

**Министерство образования и науки Кыргызской Республики
Кыргызский государственный технический университет
им. И. Раззакова**

Кафедра «Программное обеспечение компьютерных систем»

***Методические указания к выполнению лабораторных
работ по дисциплине
«Инструментальные средства разработки клиент-
серверных приложений» для студентов направления
710400 «Программная инженерия»***

Бишкек – 2024

Содержание

Введение.....	3
Лабораторная работа №1.....	4
Лабораторная работа №2.....	12
Лабораторная работа №3.....	20
Лабораторная работа №4.....	26
Лабораторная работа №5.....	29
Лабораторная работа №6.....	35

Введение

Данное пособие предназначено для студентов третьего и четвертого курсов специальности «Программная инженерия» по дисциплине «Проектирование на основе структурного шаблона (РСМЕФ)». В нем представлены рекомендации по созданию и использованию объектов при реализации клиент-серверных приложений в среде IntelliJ IDEA.

Целью выполнения лабораторных работ является:

- освоение и закрепление навыков программирования основных форм пользовательского интерфейса;
- освоение технологии создания и использования объектов «окна данных»;
- развитие навыков построения пользовательских объектов;
- освоение технологии создания приложений, работающих с базами данных;
- освоение технологии создания многодокументных приложений;
- развитие навыков построения графиков функций.

Лабораторная работа №1

“Знакомство с **IntelliJ IDEA** и первый проект”

IntelliJ IDEA — это интегрированная среда разработки (IDE) для языка программирования Java (и не только). Она предоставляет разработчикам удобный и мощный набор инструментов для создания, отладки и поддержки программного обеспечения.

Для того что бы скачать **IntelliJ IDEA** переходим на сайт разработчиков www.jetbrains.com :

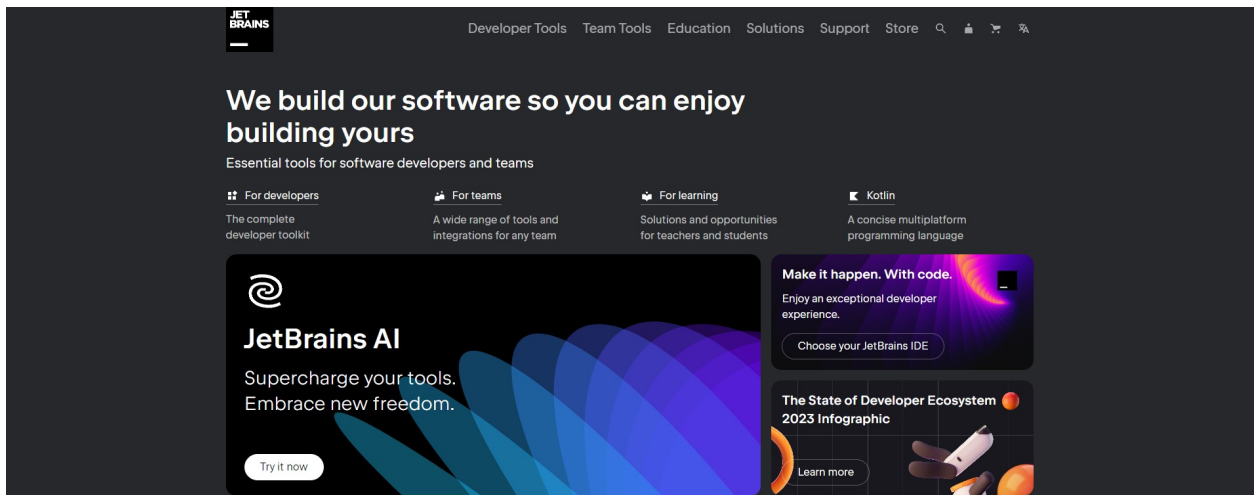


Рис. 1.1.1 сайт jetbrains.com

Нажимаем в верхнем меню баре на вкладку “Developer Tools”:

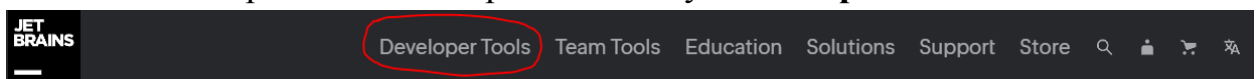


Рис. 1.1.2 Menu bar сайта jetbrains.com

Затем выбираем в выпадающем списке → **IntelliJ IDEA**:

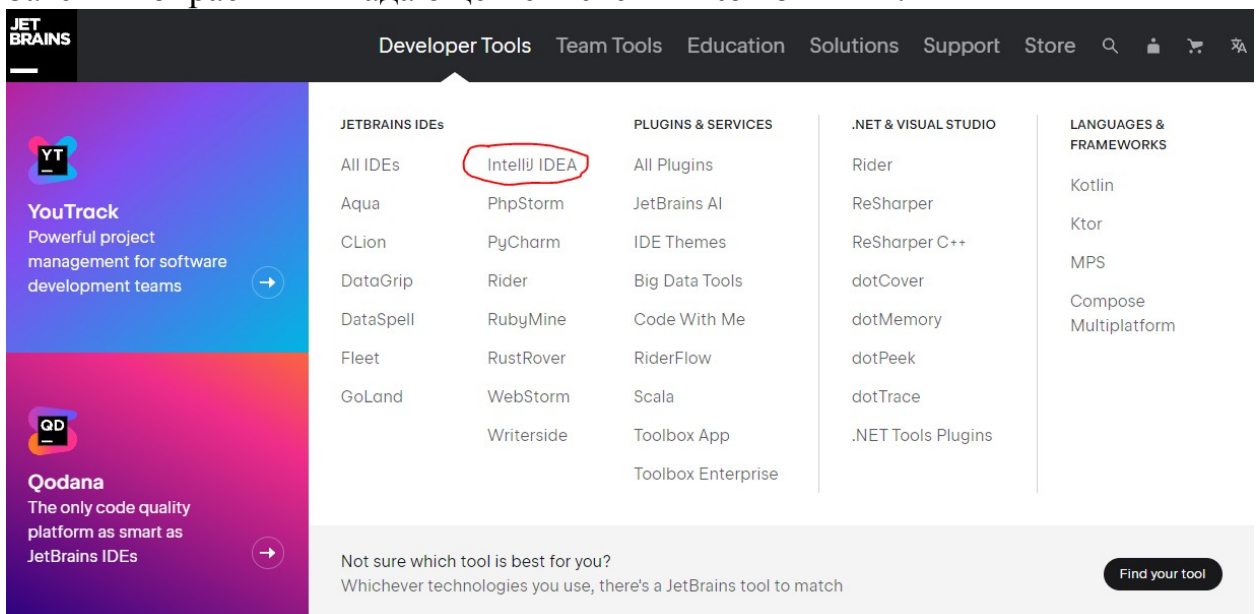


Рис. 1.1.3 Выпадающий список IDE сайта jetbrains.com

После чего нас перекидывает на страницу **IntelliJ IDEA**, где мы можем полностью ознакомиться с данной средой разработки. Но так как нам нужно **скачать** её мы нажимаем на одну из нескольких кнопок «**Download**» не важно какую:

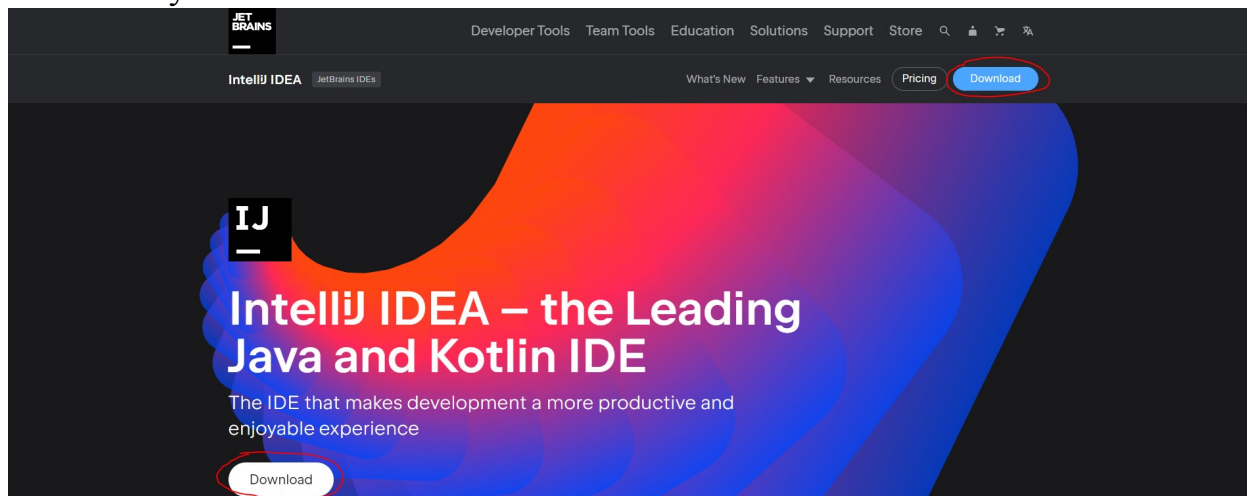


Рис. 1.1.4 страница IntelliJ IDEA

Далее у нас есть выбор между версиями данной среды разработки:

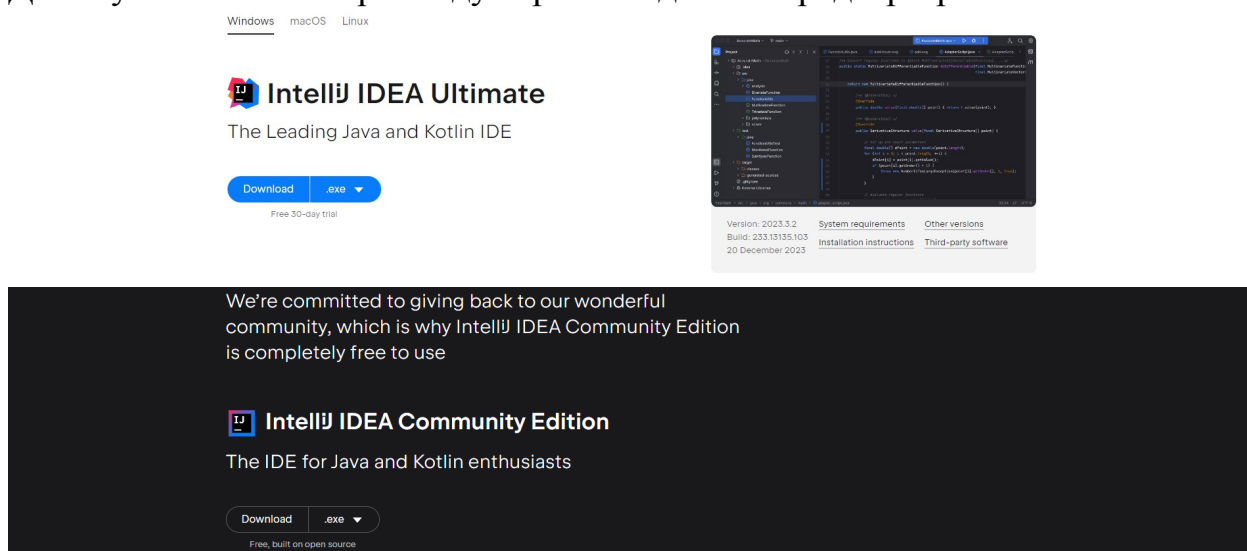


Рис. 1.1.5 разные версии IntelliJ IDEA

Примечание:

IntelliJ IDEA Community Edition

Это бесплатная версия, доступная для использования всеми пользователями. Включает в себя основные инструменты и функции для разработки на языке Java, а также поддерживает Kotlin, Groovy и Scala.

IntelliJ IDEA Ultimate Edition

Это платная версия, предоставляющая более широкий набор функций и инструментов, таких как поддержка других языков программирования (например, PHP, Python, JavaScript), интеграция с серверами приложений, инструменты для работы с базами данных, анализ кода, поддержка фреймворков и многое другое.

Для версии “Ultimate Edition” обычно дают пробный период использования на **30** дней. Поэтому её лучше скачать для изучения полного спектра функционала если вы планируете изучать и совершенствовать свои навыки программирования на языке Java.

После скачивания любой из версии устанавливаем её:

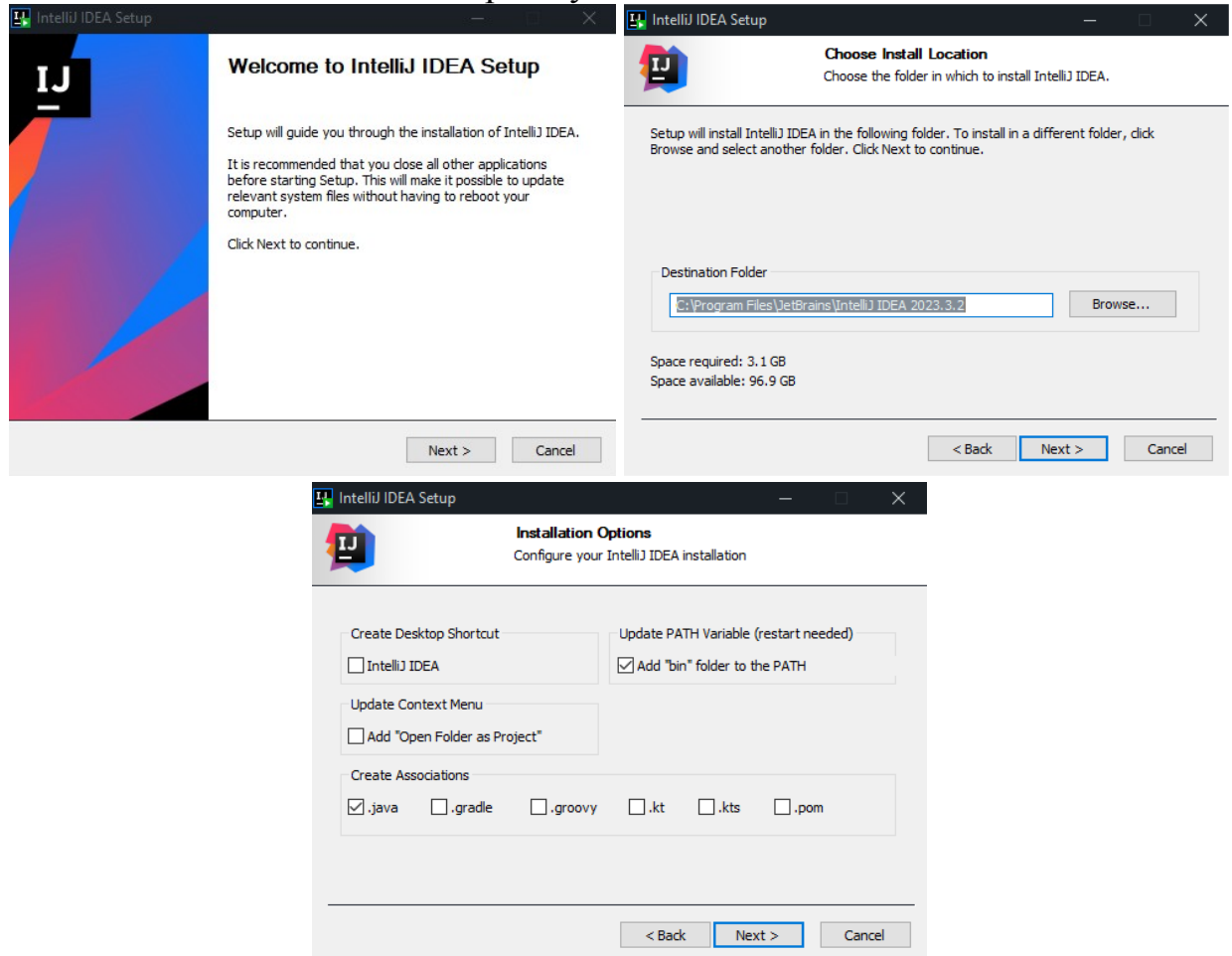


Рис. 1.1.6 установка IntelliJ IDEA

Примечание:

Лучше не менять путь установки (**Destination Folder**), дабы не было в будущем проблем с IDE.

Помимо этого галочки в окне “Опции установки (**Installation Options**)” отмечать по выбору, они сделаны лишь для удобства:

- **Create Desktop Shortcut** – создание ярлыка на рабочем столе
- **Update PATH Variable (restart needed)** – добавления пути к папке bin вашего проекта в переменную среды PATH системы. Позволит запускать проект из командной строки.
- **Add “Open Folder as Project”** – позволит открывать папки с проектами как проект даже если внутри не будет файлов связанных с IntelliJ IDEA.
- **Create Associations** – добавление ассоциации файлов для более просто и быстрого запуска их в IDE. (!!! Возможны конфликты если установлены другие IDE !!!)

Теперь можно создать проект, открываем **IntelliJ IDEA** и нажимаем на -> “**New Project**”

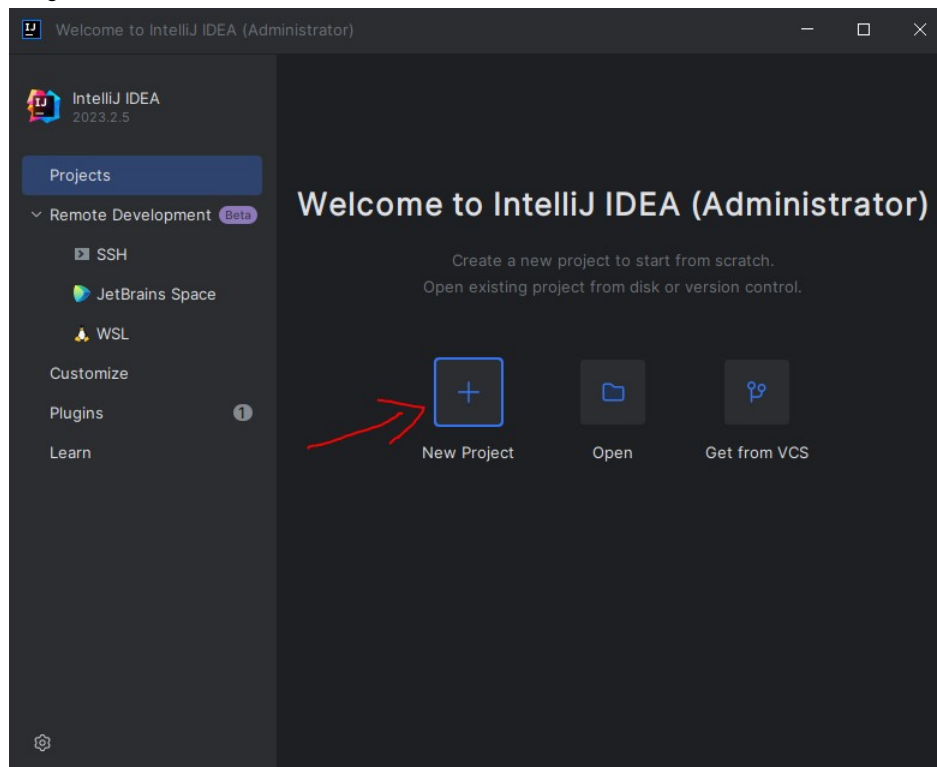


Рис. 1.2.1 начальное окно в IntelliJ IDEA

В открывшемся окне у нас автоматически будет выбрана вкладка “**New Project**” в этой вкладке настраиваем имя (**Name**), путь к проекту (**Location**), язык разработки (**Language**), систему сборки (**Build system**) и инструменты разработки Java – “**JDK**” (**Java Development Kit**).

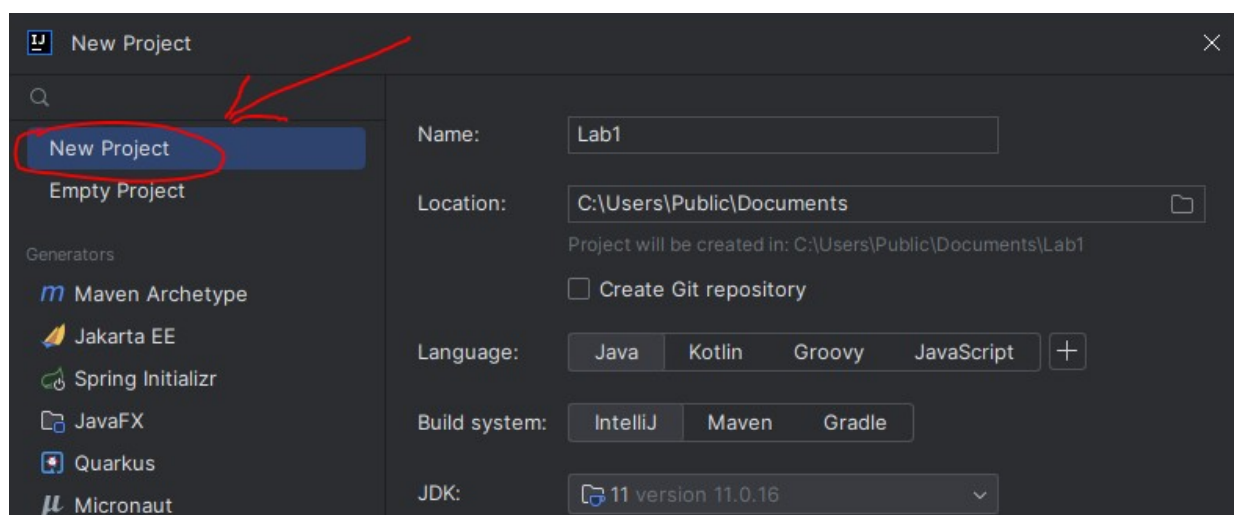


Рис. 1.2.2 создание обычного проекта в IntelliJ IDEA

Если по какой-то причине в поле **JDK** у нас пусто и нет на выбор какого-либо набора, тогда нажимаем на него ->

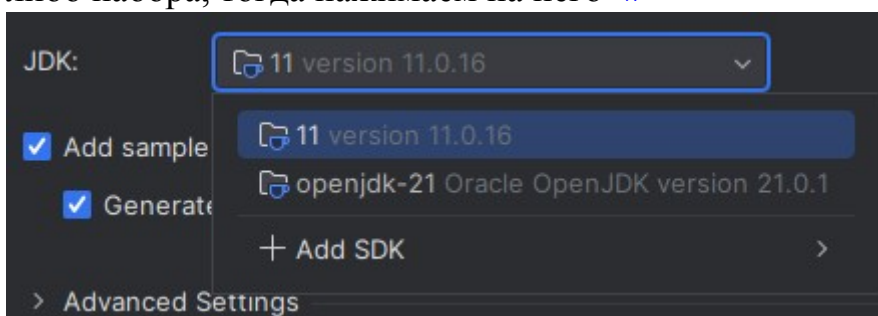


Рис. 1.2.3

Затем в открывшемся выпадающем списке нажимаем на **“+ Add SDK”** -> **“Download JDK...”** и в появившемся окошке выбираем **“Oracle OpenJDK”** ->

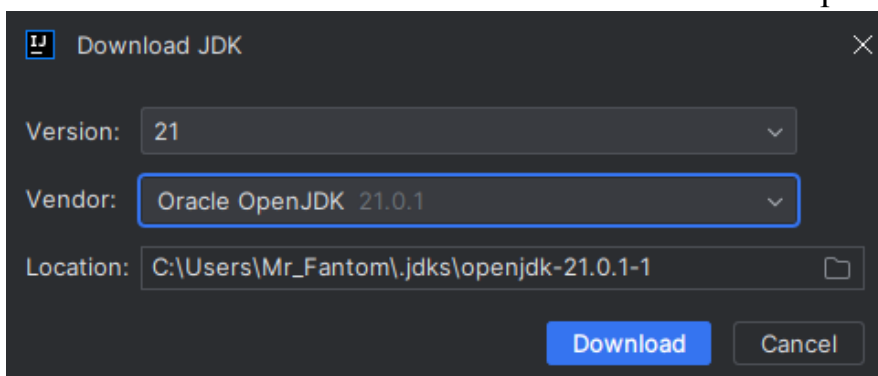


Рис. 1.2.4

И актуальную последнюю версию для стабильной работы.

После чего нажимаем на **“Download”** и у нас начнется скачивание JDK.

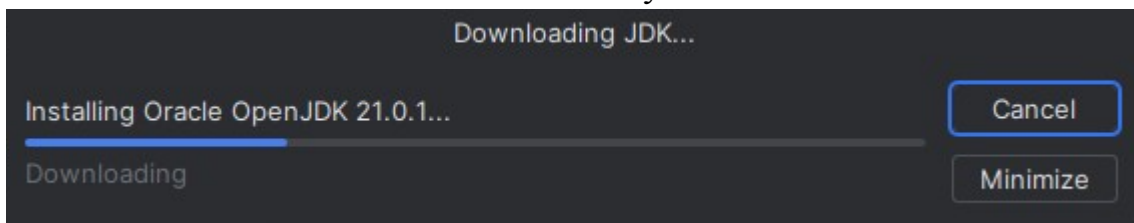


Рис. 1.2.5

Галочки на **“Add sample code”** и **“Generate code with onboarding tips”** оставляем, если они не включены -> включаем.

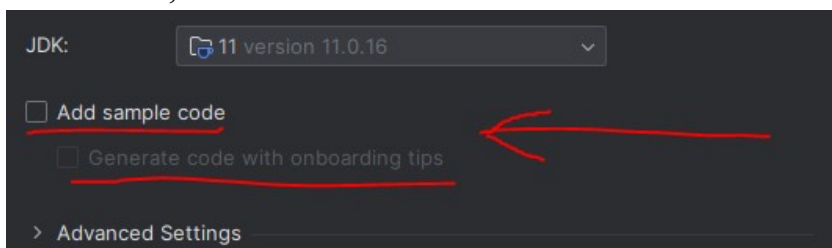


Рис. 1.2.6

После всех манипуляций можно нажать на кнопку **“Create”**. После чего откроется окно нашего проекта

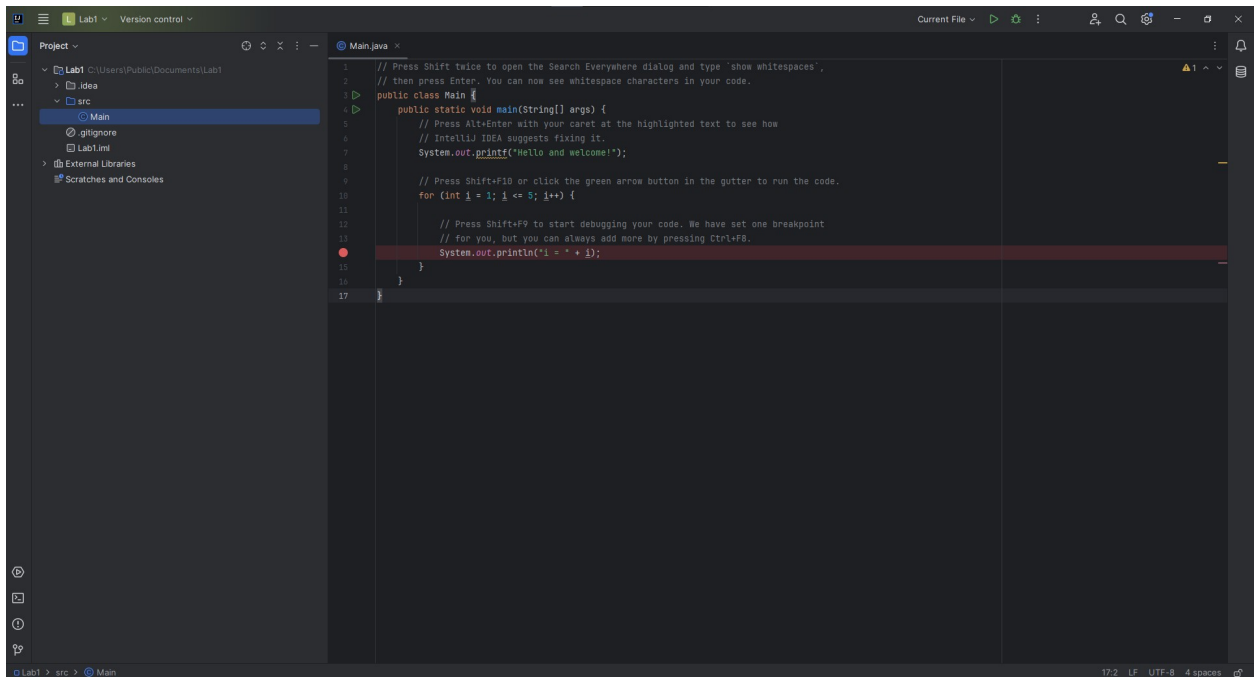


Рис. 1.2.7 Окно проекта в IntelliJ IDEA

В левой части окна проекта расположена его структура:

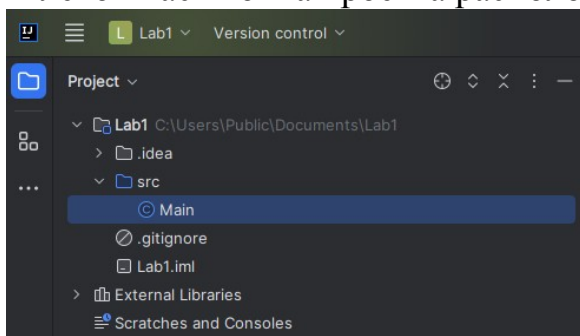


Рис. 1.2.8 Структура проекта Java

В правой части окна проекта расположена часть с кодом нашего файла/класса Main.java

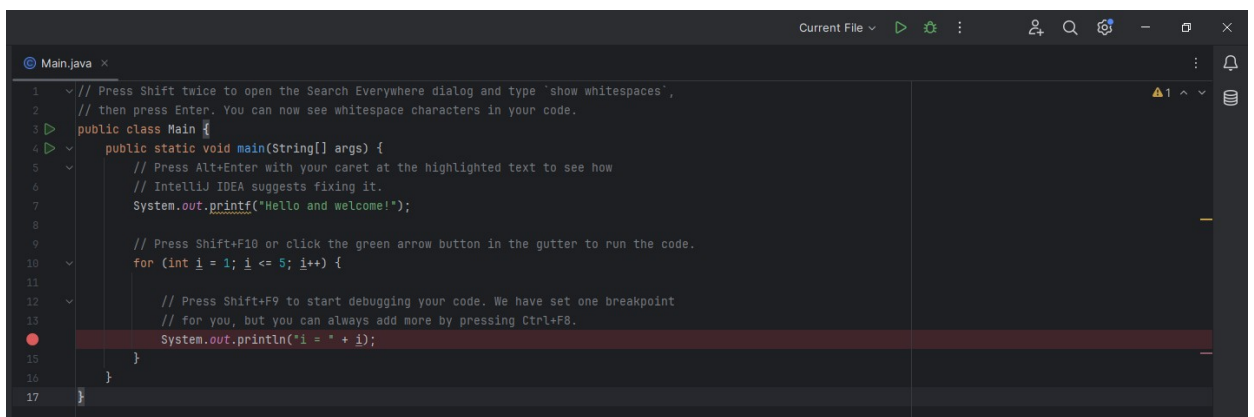
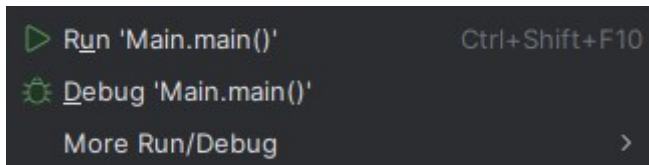


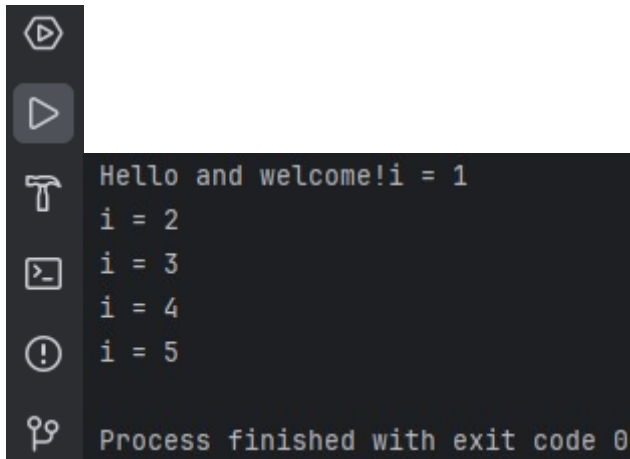
Рис. 1.2.9 Сгенерированный код Java

Который был сгенерирован благодаря галочкам “**Add sample code**” и “**Generate code with onboarding tips**”.

Так же этот код можно запустить, нажав на **ПКМ** (правая кнопка мыши) по любому месту во вкладке с кодом и в появившемся **контекстном меню** выбрать “**Run ‘Main.main()’**” или связку клавиш “**Shift+F10**” или “**Ctrl+Shift+F10**” ->



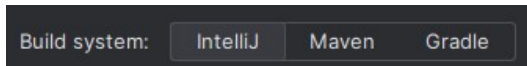
Результат кода появится снизу во вкладке **Run ->**



Поздравляю это ваш первый проект в **IntelliJ IDEA**.

Примечание:

Разница между “Системами сборки”:



Maven — это инструмент управления проектами и система сборки, которая использует XML-файлы для описания структуры проекта, зависимостей и задач сборки. Maven автоматически загружает библиотеки из центрального репозитория, и это делает процесс управления зависимостями довольно простым. Maven также может выполнять различные задачи, такие как компиляция, тестирование, создание JAR-файлов и др.

Gradle — это инструмент управления проектами и система сборки, которая использует Groovy-подобный DSL (Domain-Specific Language) для описания проекта. Gradle обладает большей гибкостью и мощностью по сравнению с Maven, что делает его предпочтительным выбором для некоторых проектов. Он также поддерживает автоматическую загрузку зависимостей и выполнение различных задач сборки.

IntelliJ — также предоставляет возможность настройки и использования собственных средств сборки проекта. Это может быть полезно в тех случаях, когда ваш проект использует нестандартные средства сборки или если у вас есть специфические требования к настройке проекта.

Для более детального знакомства с IntelliJ IDEA можно наводить курсор на разные кнопки что бы узнать, что за что отвечает, например кнопки, которые находятся снизу в левом углу:

Services (Службы):



Открывает панель "Services", где вы можете управлять различными сервисами, такими как базы данных, подключения к удаленным серверам и т.д.

Build (Сборка):



Если вы включите режим "Automatically show Build tool window" (Автоматически отображать окно инструментов сборки) в настройках IntelliJ IDEA, то при каждой сборке проекта вам будет автоматически открываться вкладка Build. Эта вкладка содержит информацию о процессе сборки, ошибки и предупреждения, если они есть.

Terminal (Терминал):



Позволяет вам взаимодействовать с командной строкой непосредственно из среды разработки. Вы можете выполнять различные команды, работать с инструментами командной строки, а также выполнять сценарии сборки.

Problems (Проблемы):



Предназначена для отображения списка проблем в вашем проекте, таких как ошибки компиляции, предупреждения и другие проблемы, выявленные статическим анализатором кода.

Version Control (Контроль версий):



Позволяет вам работать с системами контроля версий, такими как Git. Вы можете коммитить изменения, подтягивать изменения из репозитория, просматривать историю изменений и многое другое.

Так же в верхнем левом углу есть кнопка, которая открывает меню бар с нужными вкладками:



Задание к лабораторной работе:

1. Установить IntelliJ IDEA Ultimate (рекомендовано)
2. Создать и запустить свой первый java проект.
3. Изучить интерфейс новой IDE.

Лабораторная работа №2

“Создание простого окна приложения с помощью библиотеки Swing”

В **IntelliJ IDEA**, как и во многих других **IDE** есть возможность создавать приложения с помощью библиотеки **Java Swing**.

Примечание:

Java Swing:

Это библиотека для создания графических интерфейсов в стиле AWT (Abstract Window Toolkit), но более современная и расширенная. Swing предоставляет широкий набор компонентов GUI, таких как кнопки, текстовые поля, таблицы и др. Он является частью стандартной библиотеки Java.

Создаём проект как в первой лабораторной работе, только в этот раз убираем галочки с “Add sample code” и “Generate code with onboarding tips”.

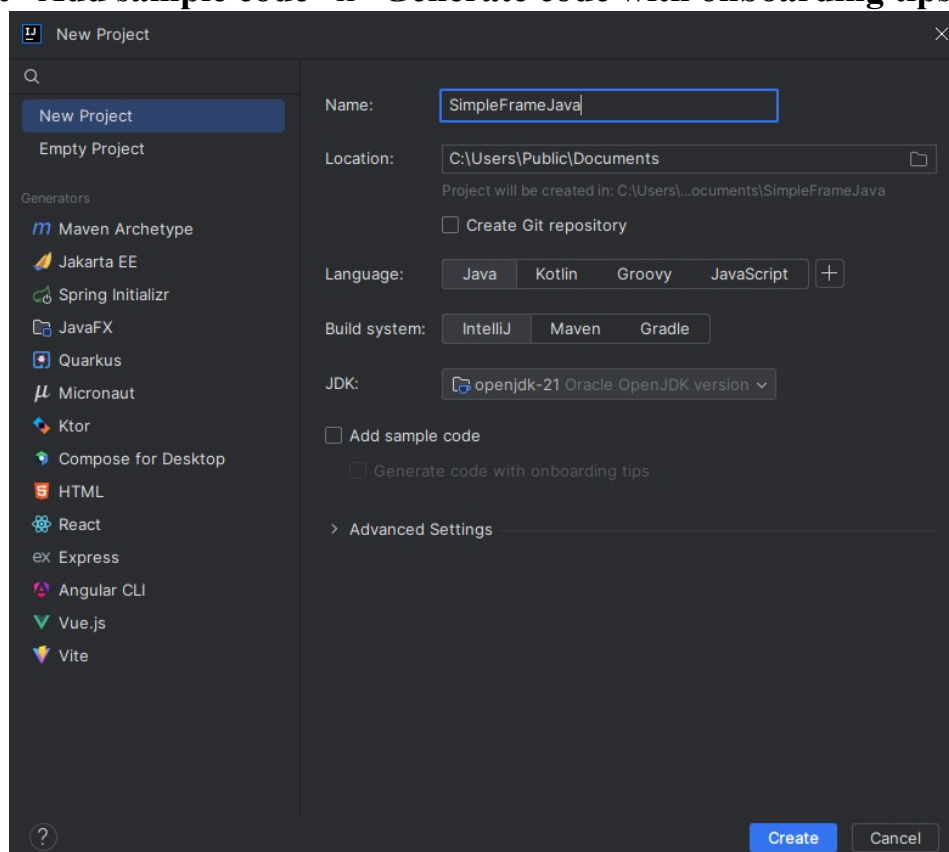


Рис. 2.1.1 Создание проекта

После создания видим, что наш проект совсем пуст:

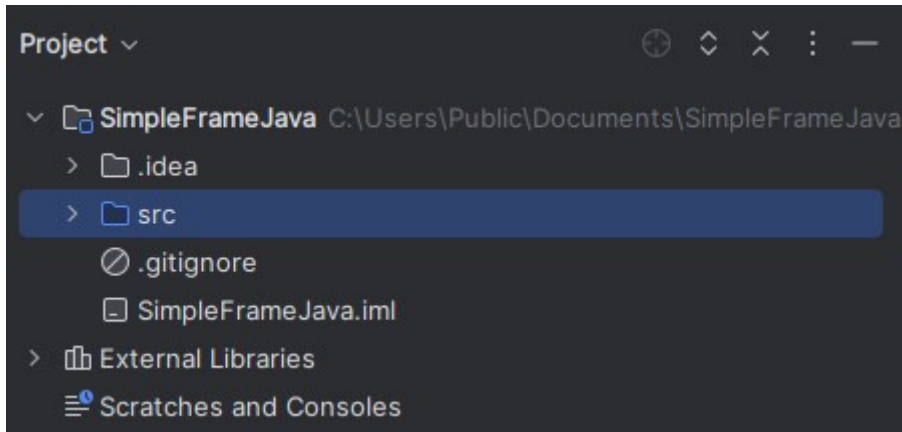


Рис. 2.1.2 Структура проекта

Поэтому нажимаем **ПКМ** (правой кнопкой мыши) по папке `src` и в контекстном меню выбираем **New -> Java Class**

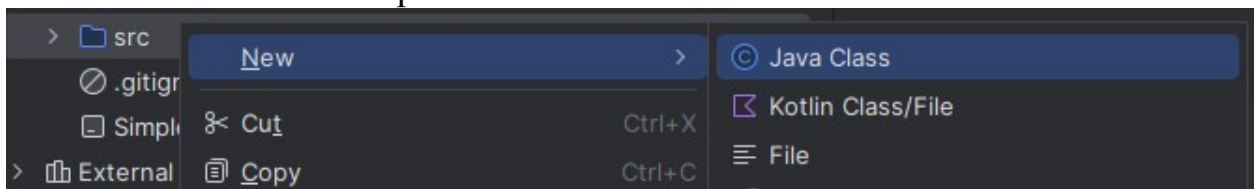


Рис. 2.1.3 Создание главного класса проекта

Называем его -> **“Main”** и нажимаем **Enter** на клавиатуре:

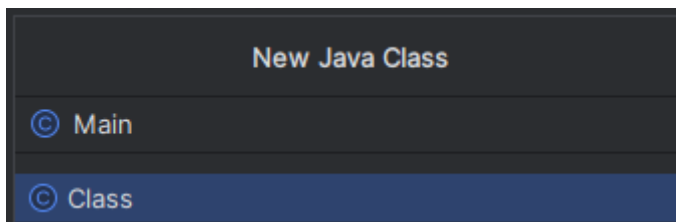


Рис. 2.1.4 Название класса

Получаем пустой класс:

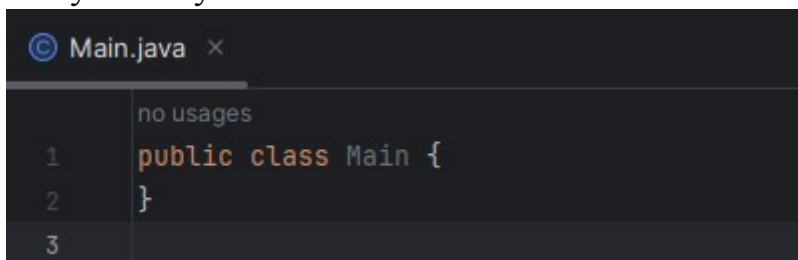
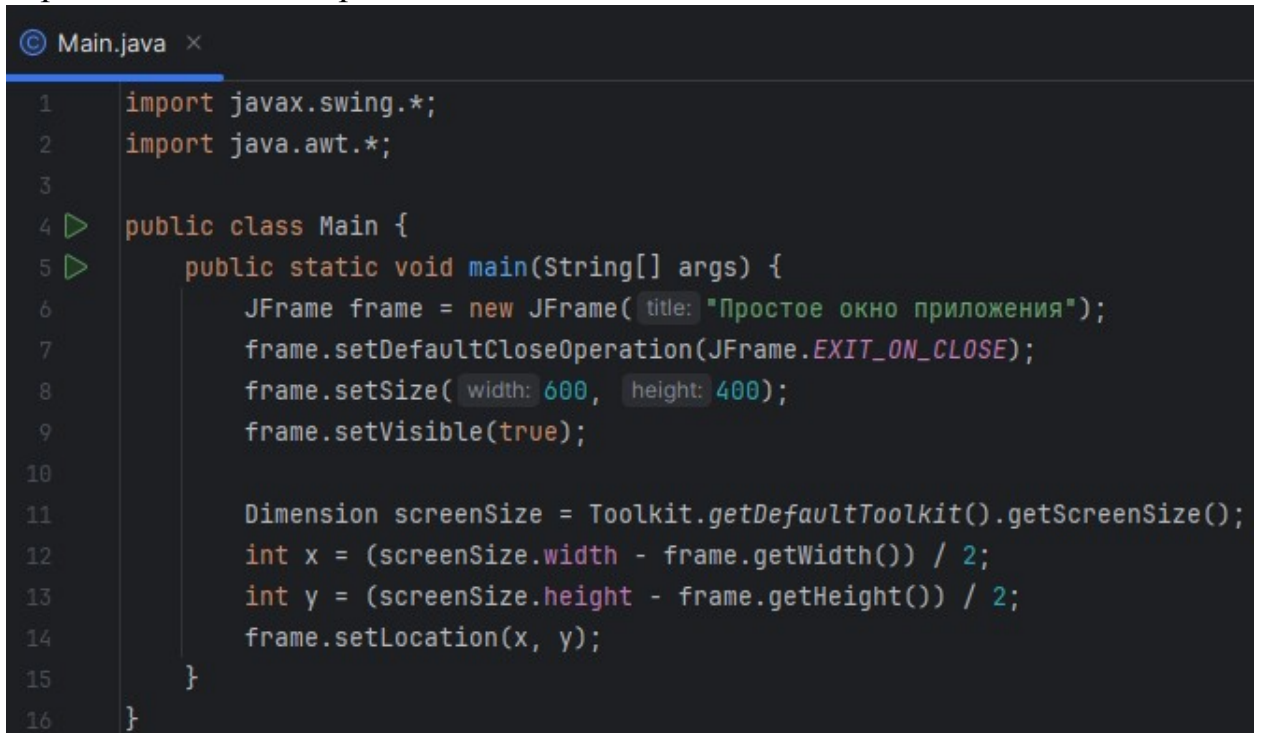


Рис. 2.1.5 Код класса

Теперь нужно импортировать пакеты и создать функцию, которая будет отрисовывать окно приложения:



```
1  import javax.swing.*;
2  import java.awt.*;
3
4  public class Main {
5      public static void main(String[] args) {
6          JFrame frame = new JFrame( title: "Простое окно приложения");
7          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
8          frame.setSize( width: 600, height: 400);
9          frame.setVisible(true);
10
11          Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
12          int x = (screenSize.width - frame.getWidth()) / 2;
13          int y = (screenSize.height - frame.getHeight()) / 2;
14          frame.setLocation(x, y);
15      }
16  }
```

Рис. 2.1.6 Готовый класса

Пояснение кода:

`import javax.swing.*;` - Импорт пакета `javax.swing`, который предоставляет классы для создания графических пользовательских интерфейсов (GUI) в Java.

`import java.awt.*;` - Импорт пакета `java.awt`, который предоставляет базовые классы для создания и управления графическими элементами и событиями в приложениях Java.

`JFrame frame = new JFrame("Простое окно приложения");` - Создание объекта `JFrame` с указанием заголовка.

`frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);` - Установка операции закрытия окна по умолчанию при нажатии на кнопку закрытия.

`frame.setSize(600, 400);` - Установка размеров окна (ширина x высота).

`frame.setVisible(true);` - Сделать окно видимым.

`Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();` - Получение размеров экрана.

`int x = (screenSize.width - frame.getWidth()) / 2;` - Вычисление координаты x для центрирования окна.

`int y = (screenSize.height - frame.getHeight()) / 2;` - Вычисление координаты y для центрирования окна.

`frame.setLocation(x, y);` - Установка координат для центрирования окна по экрану.

После запуска данного кода мы получим пустое окно нашего приложения:

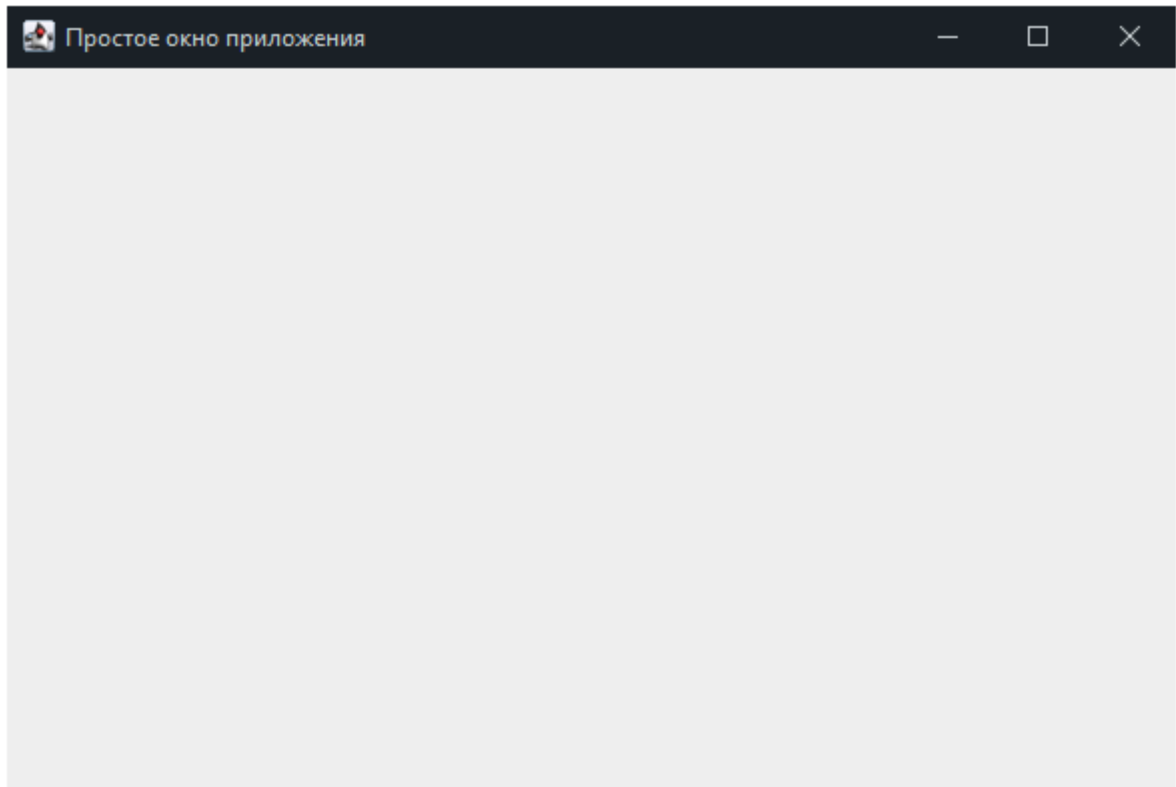


Рис. 2.1.7 Полученное окно приложения

Полный код:

```
import javax.swing.*;
import java.awt.*;

public class Main {
    public static void main(String[] args) {
        JFrame frame = new JFrame("Простое окно приложения");
        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        frame.setSize(600, 400);
        frame.setVisible(true);

        Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
        int x = (screenSize.width - frame.getWidth()) / 2;
        int y = (screenSize.height - frame.getHeight()) / 2;
        frame.setLocation(x, y);
    }
}
```

Помимо ручного кодирования мы можем использовать встроенный конструктор что бы создать форму, для этого нажимаем ПКМ по папке `src` и в контекстном меню выбираем **New -> Swing UI Designer -> GUI Form**

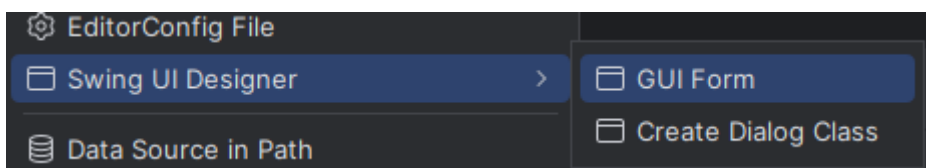


Рис. 2.2.1 Создание GUI Form

Пишем название нашей формы (**Form name**), так же можно изменить и название класса формы (**Class name**) и жмём **OK**.

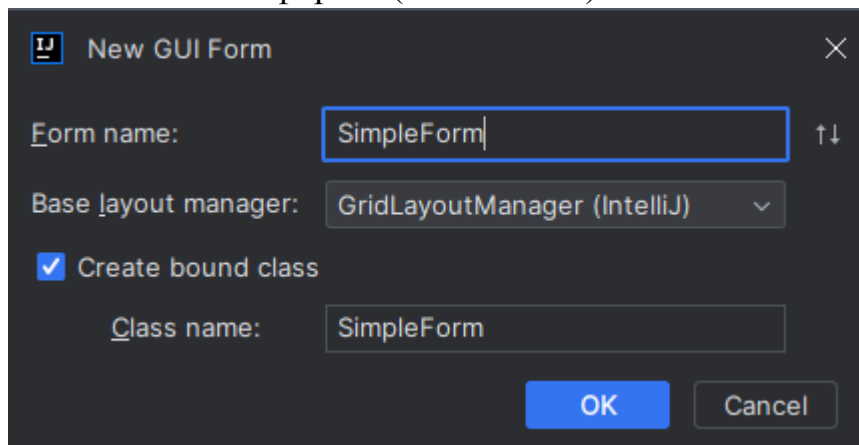


Рис. 2.2.2 Полученное окно приложения

И у нас создаётся папка с название формы, в ней создаются 2 файла: класс (**SimpleForm**) и конструктор GUI формы (**SimpleForm.form**).

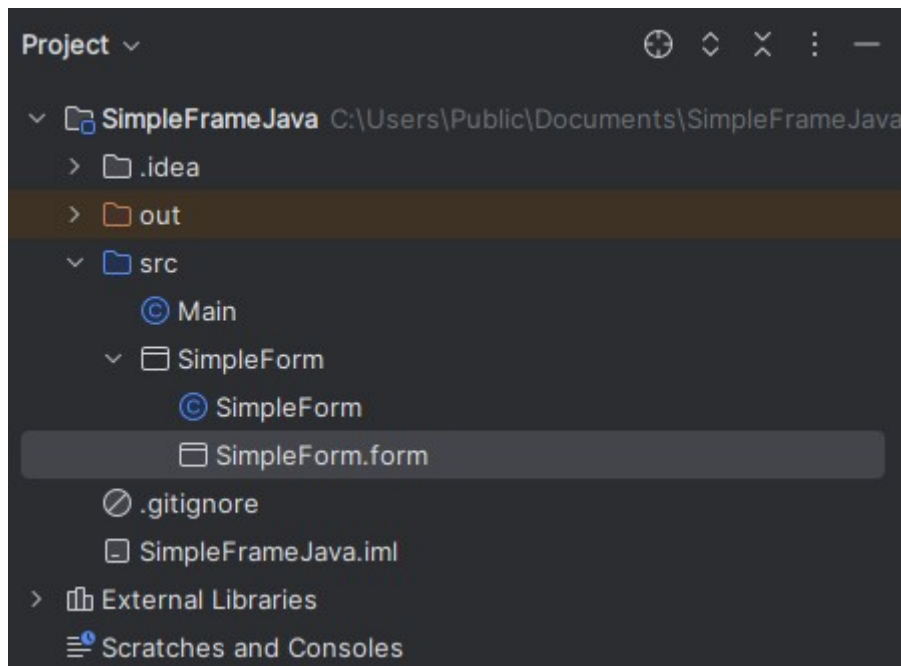


Рис. 2.2.3 Структура проекта после создания GUI Form

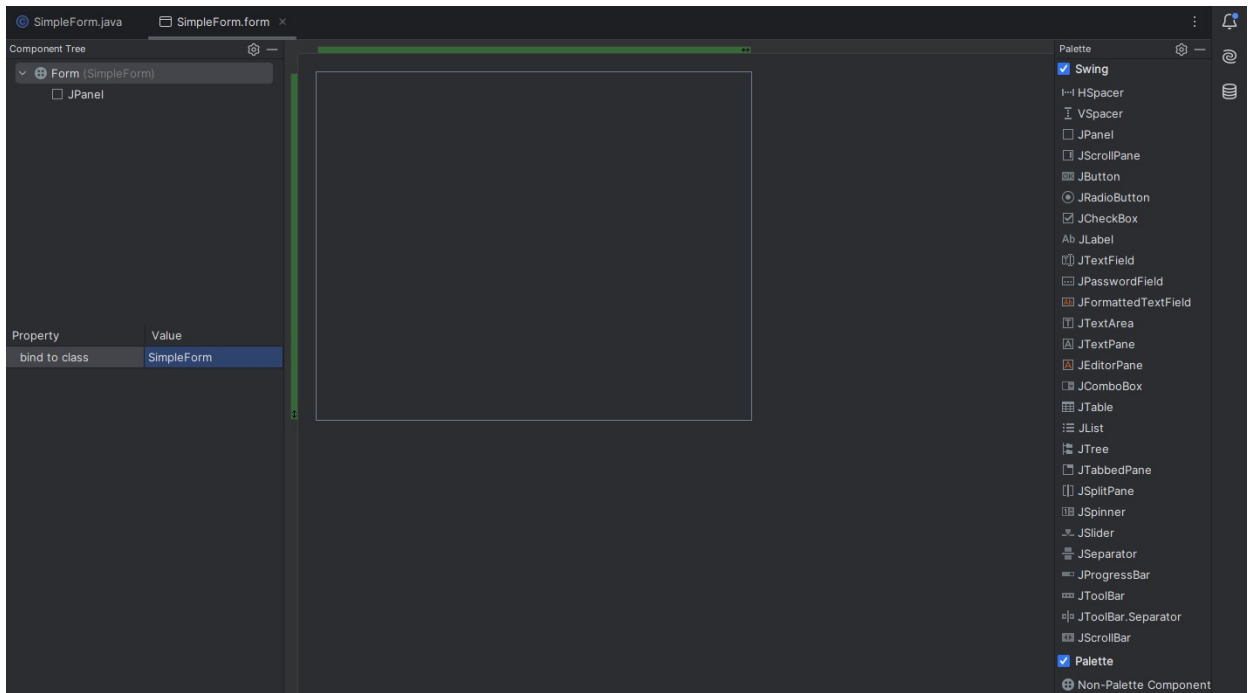


Рис. 2.2.4 Конструктор форм GUI Form

Если в конструкторе нажать на какой-либо элемент, то можно посмотреть и отредактировать его свойства.

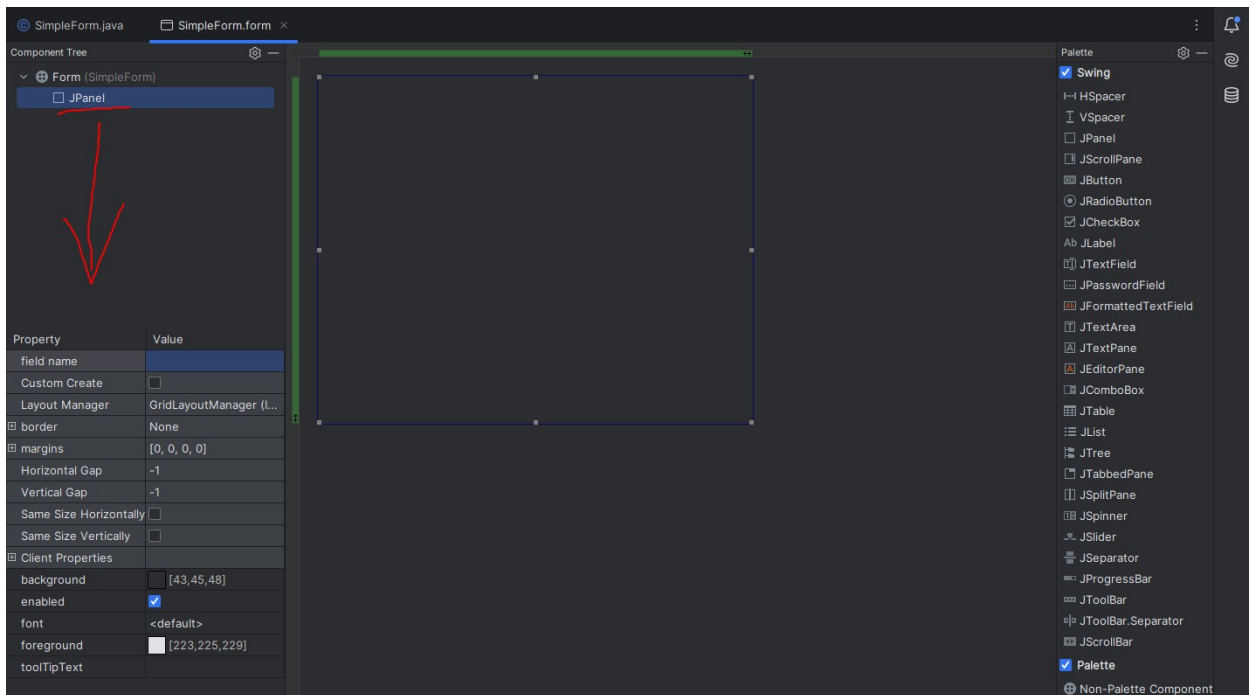
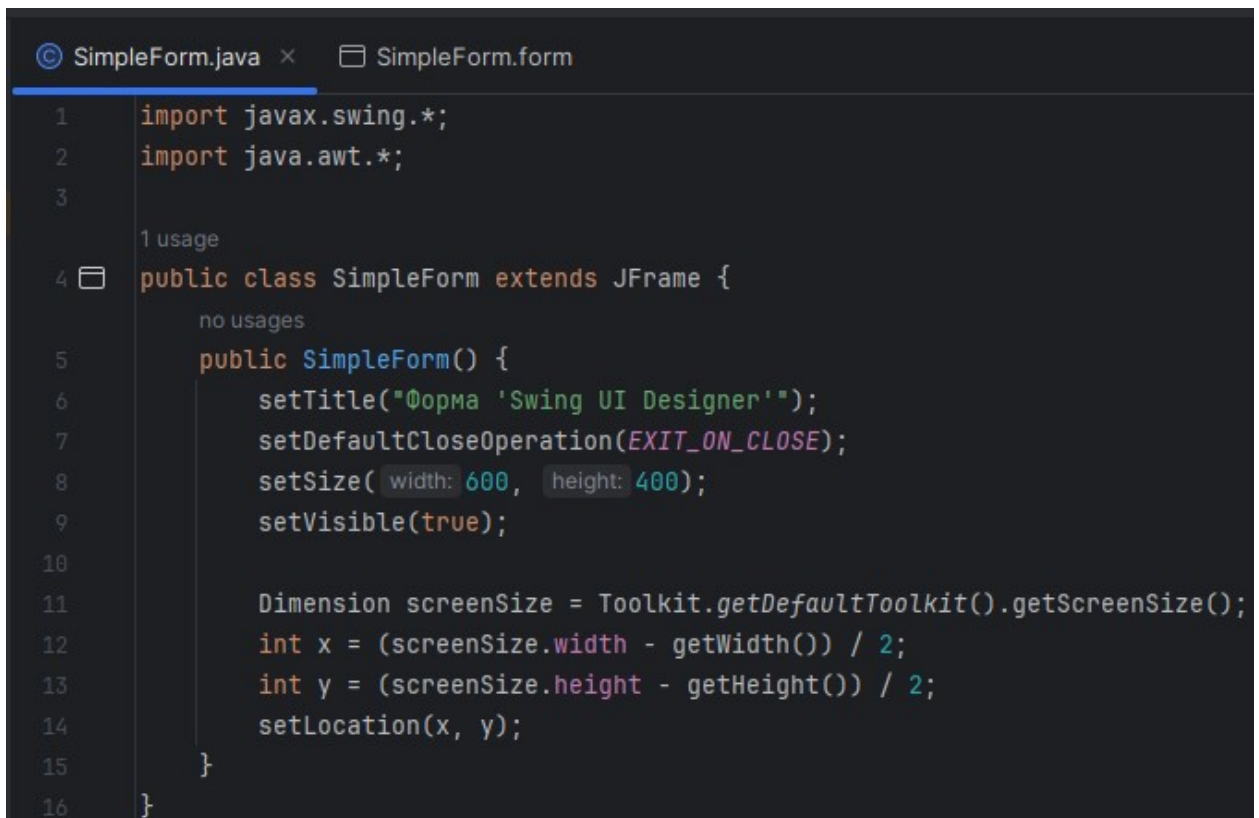


Рис. 2.2.5 Свойства элемента на GUI Form

Так как после создания наш класса пустой, нам нужно определить его.

Для этого нам нужно с помощью **extends** расширить (наследовать) функциональность от класса **JFrame**.

Затем нам нужно создать конструктор нашего класса для того, чтобы в нём задать базовые свойства нашей формы ->



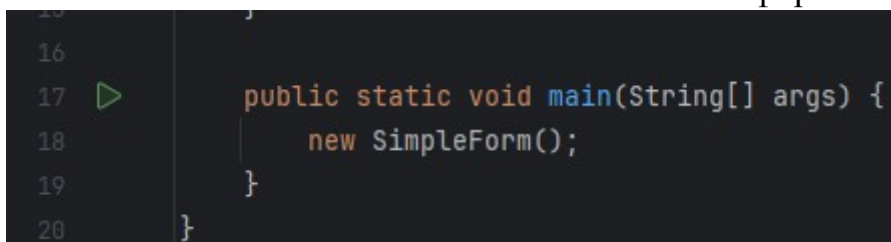
```
1 import javax.swing.*;
2 import java.awt.*;
3
4 public class SimpleForm extends JFrame {
5     public SimpleForm() {
6         setTitle("Форма 'Swing UI Designer'");
7         setDefaultCloseOperation(EXIT_ON_CLOSE);
8         setSize(width: 600, height: 400);
9         setVisible(true);
10
11         Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
12         int x = (screenSize.width - getWidth()) / 2;
13         int y = (screenSize.height - getHeight()) / 2;
14         setLocation(x, y);
15     }
16 }
```

Рис. 2.2.6 Создание конструктора нашей формы

Как видно со скриншота в конструкторе не нужно создавать экземпляра, а сразу можно настроить все свойства формы.

Но запустить форму все равно нельзя, так как мы **не инициализировали** экземпляра нашей формы.

Для того что бы инициализировать экземпляр нашей формы нам нужно создать метод: **отступаем от нашего конструктора одну строчку и пишем `psvm` затем нажимаем `Enter`** и у нас появляется метод, в теле этого метода мы пишем ключевое слово **new** и название нашей формы.



```
16
17 public static void main(String[] args) {
18     new SimpleForm();
19 }
20 }
```

Рис. 2.2.7 Метод для инициализации нашей формы

Полный код нашей формы:

```
import javax.swing.*;
import java.awt.*;

public class SimpleForm extends JFrame {
    public SimpleForm() {
        setTitle("Форма 'Swing UI Designer'");
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setSize(600, 400);
        setVisible(true);

        Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
        int x = (screenSize.width - getWidth()) / 2;
        int y = (screenSize.height - getHeight()) / 2;
        setLocation(x, y);
    }

    public static void main(String[] args) {
        new SimpleForm();
    }
}
```

После чего запускаем наш проект и у нас появляется наше окно приложения.

Результат:

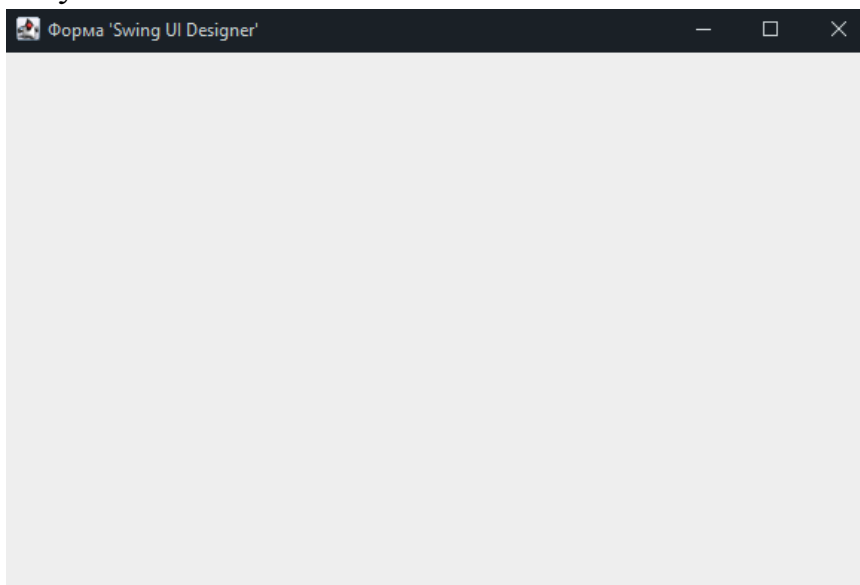


Рис. 2.2.8 Инициализированная форма

Задание к лабораторной работе:

1. Создать окно с помощью библиотеки Swing.
2. Создать окно с помощью конструктора форм “Swing UI Designer”.
3. Изучить интерфейс конструктора “Swing UI Designer”.
4. Изучить свойства класса JFrame (самостоятельно).

Лабораторная работа №3

“Изучение элементов интерфейса окон приложения Swing”

В библиотеке **Swing** существует множество элементов, позволяющих разнообразить и улучшить визуальную составляющую программ. Среди таких элементов выделяются **JTextField**, **JLabel** и **JButton**, **JRadioButton**, **JComboBox** которые являются ключевыми для взаимодействия с пользователем. В данной лабораторной работе мы глубже погрузимся в работу с этими элементами, создав приложение, где пользователь может вводить текст, а приложение в ответ отобразит его на главном окне.

Создаем проект как в прошлой лабораторной работе либо используем тот же проект. Для создания формы будет использовать **Swing UI Designer**.

После создания формы переходим в конструктор форм, с права будет весь возможный список элементов интерфейса (**Palette**).

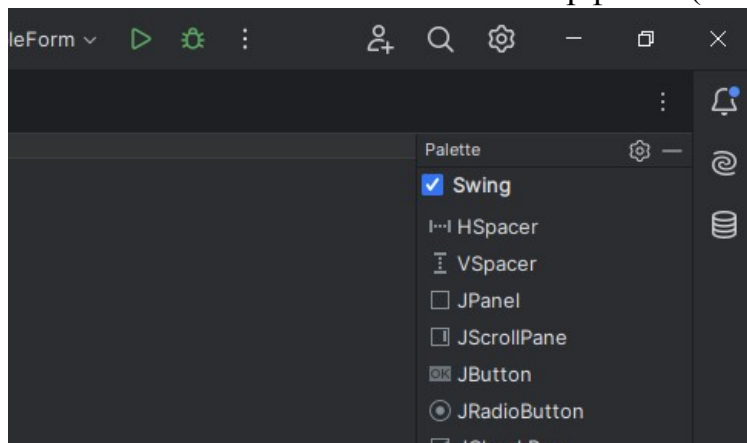


Рис. 3.1.1 Список элементов

Для того что бы добавить нужные элементы в конструкторе нам нужно перетащить интересующий нас элемент на форму конструктора.

После чего как упоминалось ранее в предыдущей лабораторной, у нас есть возможность настраивать элементы (их свойства).

Property	Value
field name	panel1
Custom Create	☐
Layout Manager	GridLayoutManager (l...
☒ border	None
type	None
title	
title justification	Default
title position	Default
title font	
title color	

Рис. 3.1.2 Список свойств

У каждого элемента, может быть, разнообразное количество свойств, которые можно настроить как в данном разделе, так и в самом коде позже, разница лишь в том, что, если их указывать изначально до инициализации, все свойства будут сохраняться в отдельный файл, а если их изменять после инициализации то их свойства будут храниться в коде, что не всегда хорошо.

Примечание:

Основные свойства элементов интерфейса которые будут использоваться:
fieldname – название элемента в коде, по которому можно будет обращаться к этому элементу;

Layout Manager – (менеджер компоновки) в Java Swing - это механизм, который управляет распределением и размещением компонентов;

text – это свойство, обозначающее текстовое содержимое компонента;

maximumSize – максимальные размеры компонента;

minimumSize – минимальные размеры компонента;

preferredSize – предпочтительные размеры компонента;

visible – видимость компонента (true или false).

Более подробно про компоненты и их свойства можно почитать:

<https://docs.oracle.com/javase/tutorial/uiswing/>

И так после того, как мы научились добавлять элементы делаем примерно такую форму:

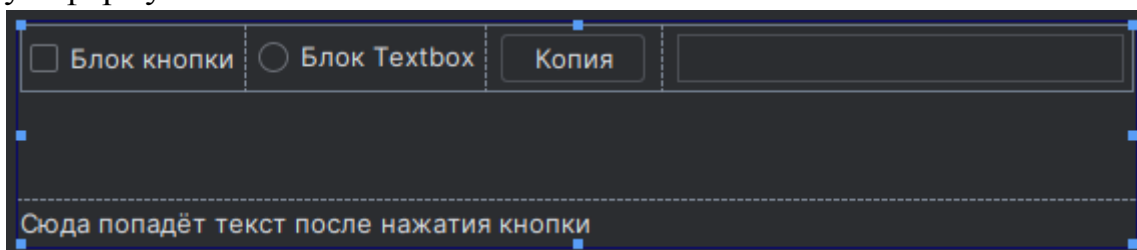


Рис. 3.1.3 Форма для ознакомления

Для **JButton** и **JTextField** свойства **enabled** убираем галочку:



А дерево компонентов должно будет выглядеть и называться примерно так:

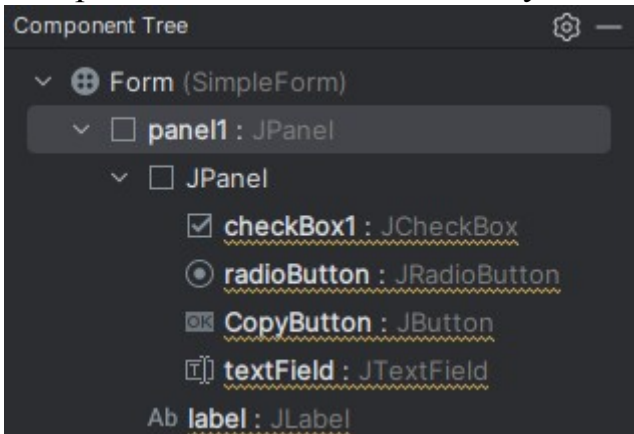


Рис. 3.1.4 Дерево компонентов

Расположение, визуальные свойства можно делать как удобно.

Затем вбиваем код:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;

public class SimpleForm extends JFrame {

    private JPanel panell;
    private JCheckBox checkBox1;
    private JRadioButton radioButton;
    private JButton CopyButton;
    private JTextField textField;
    private JLabel label;

    public SimpleForm() {
        setTitle("Форма 'Swing UI Designer'");
        setContentPane(panell);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setSize(600, 200);
        setVisible(true);
        Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();
        int x = (screenSize.width - getWidth()) / 2;
        int y = (screenSize.height - getHeight()) / 2;
        setLocation(x, y);
        CopyButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                if (radioButton.isSelected()) {
                    String text = textField.getText();
                    if (!text.isEmpty()) {
                        label.setText(text);
                    }
                } else {
                    label.setText("TextField пуст!");
                }
            }
        });
        radioButton.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                textField.setEnabled(radioButton.isSelected());
            }
        });
        checkBox1.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                CopyButton.setEnabled(checkBox1.isSelected());
            }
        });
    }

    public static void main(String[] args) {
        new SimpleForm();
    }
}
```

И мы получаем вот такую форму:

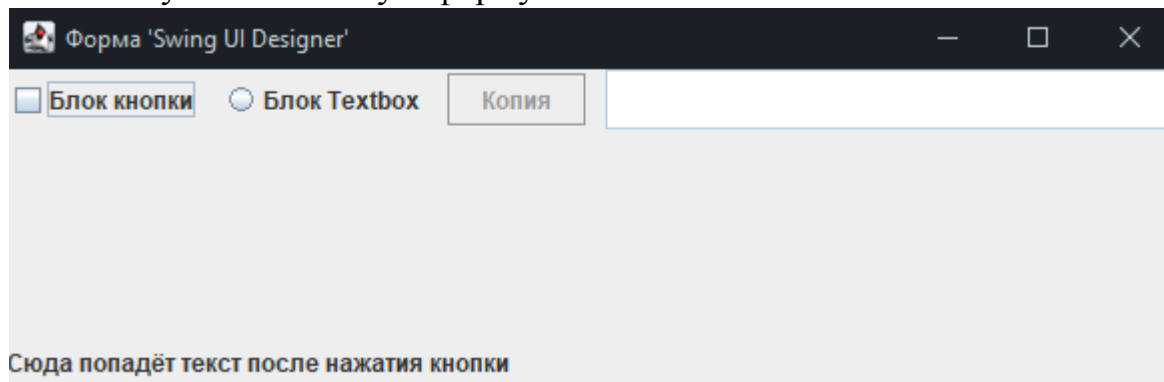


Рис. 3.1.5 Итоговая форма

Логика работы очень проста, при нажатии на “**Блок кнопки (JCheckBox)**” у нас будет *блокироваться/разблокироваться* кнопка “**Копия (JButton)**”, при нажатии “**Блок Textbox (JRadioButton)**” будет *блокироваться/разблокироваться* “**JTextField**”, если в поле для текста нет какого-либо текста, то в **JLabel** будет появляться надпись “**TextField пуст!**” нужно заполнить сначала поле, если поле заполнено, то текст будет вставляться в “**JLabel**”.

Как видно из кода для того, чтобы сделать метод для какого-либо элемента, нам нужно добавить **Listener** (слушатель), после чего переопределить стандартный метод, предоставляемый нам библиотекой. И в теле метода расписать то, что нам требуется выполнить при взаимодействии с элементом.

Помимо кода, мы можем добавить какой-либо метод через конструктор, для этого нам нужно нажать **ПКМ** по элементу для которого мы хотим добавить метод затем выбрать **Create Listener**:

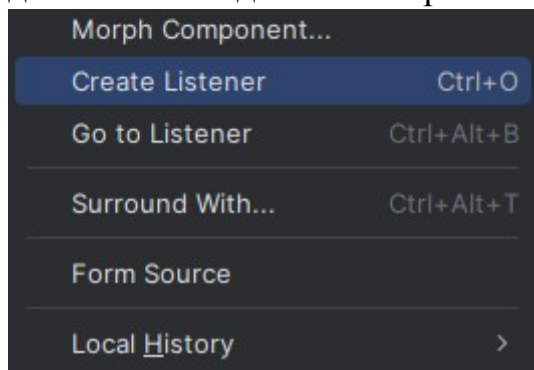


Рис. 3.1.6 Создание Listener

После чего в появившемся окне выбрать нужный нам **Listener** (слушатель).

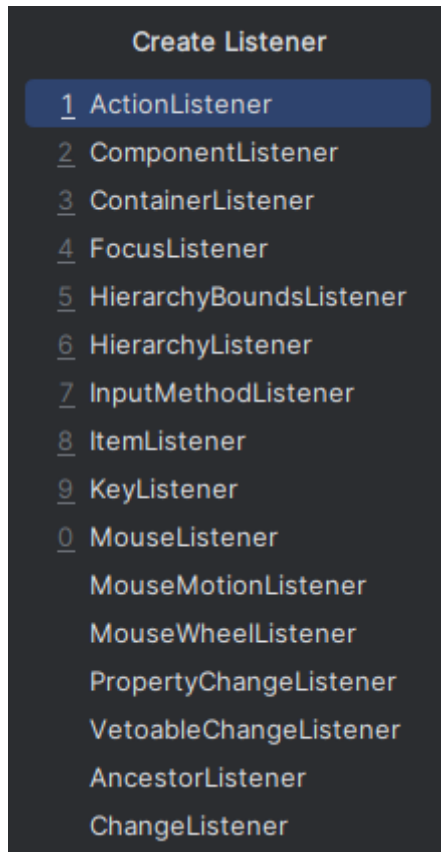


Рис. 3.1.7 Доступные Listener'ы

Примечание:

В Java существует механизм слушателей, предоставляющий способ реагировать на события, происходящие в приложении. Слушатели событий являются интерфейсами, и объекты, реализующие эти интерфейсы, становятся "слушателями" для соответствующих событий. Когда происходит событие, связанное с определенным интерфейсом, вызываются методы слушателя.

ActionListener:

Срабатывает при действии пользователя, таком как нажатие кнопки. Метод `actionPerformed()` вызывается при возникновении события.

ComponentListener:

Слушает события, связанные с компонентами GUI (например, изменение размера компонента). Методы включают `componentResized`, `componentMoved`, `componentShown`, `componentHidden`.

ContainerListener:

Реагирует на события, связанные с контейнерами компонентов. Методы включают `componentAdded` и `componentRemoved`.

FocusListener:

Срабатывает при изменении фокуса компонента. Методы включают `focusGained` и `focusLost`.

HierarchyBoundsListener:

Отслеживает изменения границ компонентов в иерархии контейнера. Методы включают ancestorMoved и ancestorResized.

HierarchyListener:

Слушает события, связанные с иерархией компонентов. Методы включают hierarchyChanged.

InputMethodListener:

Обработывает события ввода метода. Методы включают caretPositionChanged и inputMethodTextChanged.

ItemListener:

Срабатывает при изменении состояния элемента управления, такого как флажок или кнопка. Метод itemStateChanged вызывается при изменении состояния.

KeyListener:

Реагирует на нажатие клавиш на клавиатуре. Методы включают keyTyped, keyPressed, keyReleased.

MouseListener:

Обработывает события мыши. Методы включают mouseClicked, mousePressed, mouseReleased, mouseEntered, mouseExited.

MouseMotionListener:

Реагирует на движение мыши. Методы включают mouseMoved и mouseDragged.

MouseWheelListener:

Срабатывает при вращении колеса мыши. Метод mouseWheelMoved вызывается при таких событиях.

PropertyChangeListener:

Слушает изменения свойств объекта. Метод propertyChange вызывается при изменении свойства.

VetoableChangeListener:

Реагирует на попытку изменить свойство объекта. Метод vetoableChange вызывается перед фактическим изменением.

AncestorListener:

Отслеживает изменения в иерархии предков компонента. Методы включают ancestorAdded, ancestorRemoved, ancestorMoved.

ChangeListener:

Срабатывает при изменении состояния объекта, например, в JSlider. Метод stateChanged вызывается при изменении состояния.

Задание к лабораторной работе:

1. Создать окно с элементами интерфейса из лабораторной работы.
2. Создать окно с некоторыми другими элементами из списка - изучите их.
3. Изучить свойства разных элементов интерфейса.
4. Изучить и протестировать некоторые из Слушателей.
 - 1.

Лабораторная работа №4

“Диалоговые окна. Создание диалоговых окон Swing”

В **Java** для создания диалоговых окон обычно используются классы из пакета **javax.swing.JOptionPane** и классы, предоставляемые библиотекой **AWT/Swing**.

В библиотеке **Swing** есть класс **JOptionPane** у которого есть свои собственные методы: *showMessageDialog*, *showConfirmDialog* и *showInputDialog*.

Каждый метод из которых нужен в своих определенных случаях.

Например:

showMessageDialog

```
JOptionPane.showMessageDialog(Main.this, "Hello World", "Сообщение", JOptionPane.INFORMATION_MESSAGE);
```

Используется для отображения обычных сообщений, которые не требуют ввода или подтверждения действий.

showConfirmDialog

```
int result = JOptionPane.showConfirmDialog(Main.this, "Вопрос?", "Название окна", JOptionPane.YES_NO_OPTION);
```

Используется для отображения диалоговых окон с подтверждением какого-либо действия. **Возвращает значение!**

После чего требуется проверка того, что выбрал пользователь:

```
if (result == JOptionPane.YES_OPTION) {  
    /*ДА*/  
} else {  
    /*НЕТ*/  
}
```

showInputDialog

```
String input = JOptionPane.showInputDialog(Main.this, "Введите текст:", "Ввод данных", JOptionPane.QUESTION_MESSAGE);
```

Используется для ввода каких-либо данных например: **САРТСНА**.

Возвращает значение!

Для просмотра всех вариантов создадим проект. После создания проекта создаём класс **Main** и вставляем данный код:

```
import javax.swing.*;  
import java.awt.*;  
import java.awt.event.ActionEvent;  
import java.awt.event.ActionListener;  
  
public class Main extends JFrame {  
  
    public Main() {  
        setTitle("Dialog Example");  
        setSize(300, 200);  
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
  
        Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();  
        int x = (screenSize.width - getWidth()) / 2;
```

```

int y = (screenSize.height - getHeight()) / 2;
setLocation(x, y);

JButton messageButton = new JButton("Show Message Dialog");
JButton confirmButton = new JButton("Show Confirm Dialog");
JButton inputButton = new JButton("Show Input Dialog");

messageButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(Main.this, "Привет, это
диалоговое окно!", "Сообщение", JOptionPane.INFORMATION_MESSAGE);
    }
});

confirmButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        int result = JOptionPane.showConfirmDialog(Main.this, "Вы
уверены?", "Подтверждение", JOptionPane.YES_NO_OPTION);
        if (result == JOptionPane.YES_OPTION) {
            JOptionPane.showMessageDialog(Main.this, "Вы выбрали
'Да'");
        } else {
            JOptionPane.showMessageDialog(Main.this, "Вы выбрали
'Нет'");
        }
    }
});

inputButton.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        String input = JOptionPane.showInputDialog(Main.this,
"Введите что-то:", "Ввод данных", JOptionPane.QUESTION_MESSAGE);
        if (input != null) {
            JOptionPane.showMessageDialog(Main.this, "Вы ввели: " +
input);
        } else {
            JOptionPane.showMessageDialog(Main.this, "Вы нажали
'Отмена'");
        }
    }
});

setLayout(new FlowLayout());
add(messageButton);
add(confirmButton);
add(inputButton);
setVisible(true);
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            new Main();
        }
    });
}
}

```

После запуска данного кода будет результат:

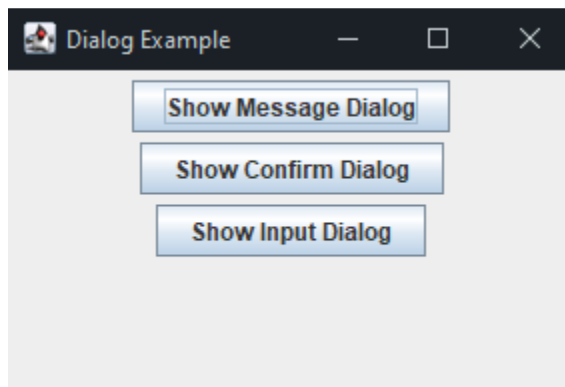


Рис. 4.1.1 Получившаяся форма

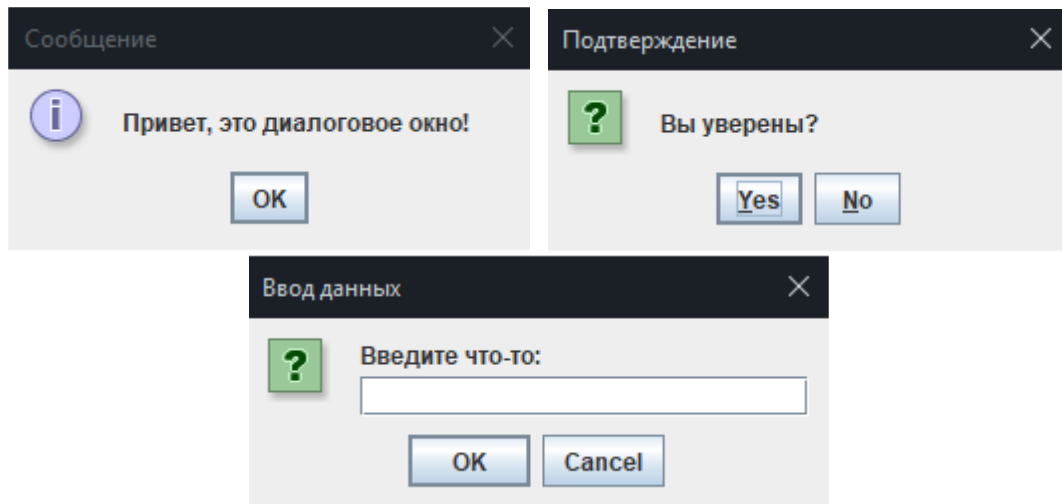


Рис. 4.1.2 Диалоговые окна от А-Я

Примечание:

У каждого диалогового окна могут быть свои перегрузки и каждое диалоговое окно может принимать своё количество параметров.

В `JOptionPane` есть различные опции для типов сообщений (message types). Вот некоторые из них:

`JOptionPane.ERROR_MESSAGE` - Сообщение об ошибке.

`JOptionPane.INFORMATION_MESSAGE` - Информационное сообщение.

`JOptionPane.WARNING_MESSAGE` - Предупреждение.

`JOptionPane.QUESTION_MESSAGE` - Вопрос.

`JOptionPane.PLAIN_MESSAGE` - Простое текстовое сообщение без иконки.

Можно использовать эти константы вместе с методами `JOptionPane` для уточнения типа сообщения, которое нужно отобразить.

Задание к лабораторной работе:

1. Создать окно с 3мя кнопка которое выводит разные диалоговые окна.
2. Протестировать различные опции для типа сообщений (message types).
3. Поэкспериментировать с диалоговыми окнами.

Лабораторная работа №5

“Создание настраиваемого диалогового окна Swing UI Designer”

В **IntelliJ IDEA** есть возможность создать своё настраиваемое диалоговое окно. Создаем проект пустой. Затем нажимаем **ПКМ** на папку **src** после чего **New -> Swing UI Designer -> Create Dialog Class**

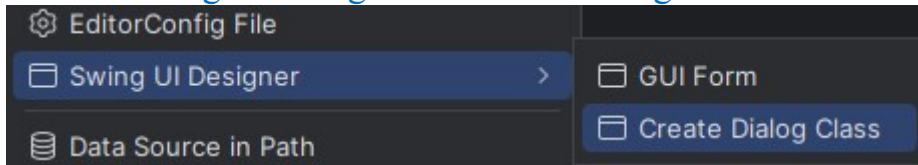


Рис. 5.1.1 Создание настраиваемого диалогового окна

После чего у нас создается готовая форма диалогового окна и код:

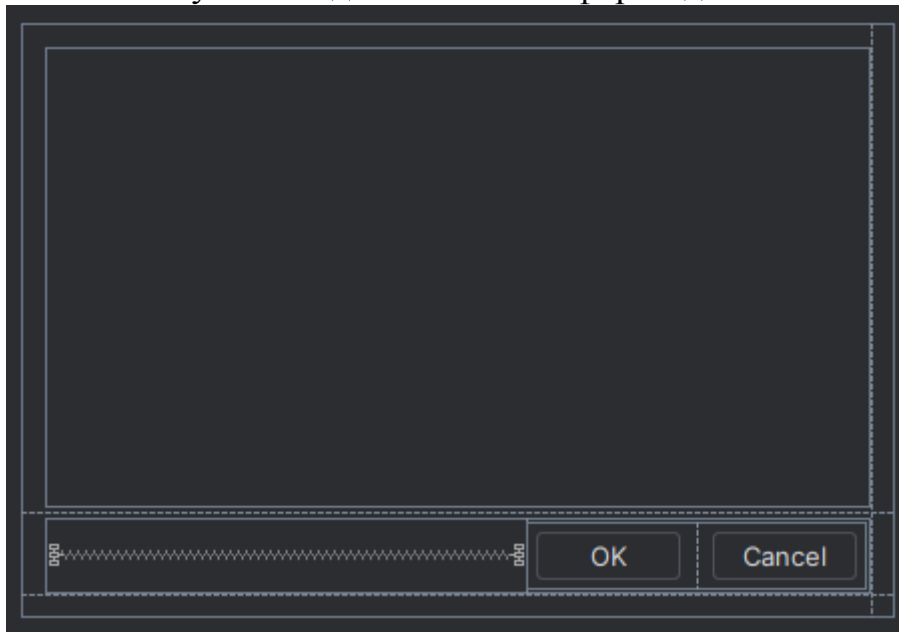


Рис. 5.1.2 Готовая форма

Код:

```
import javax.swing.*;
import java.awt.event.*;

public class CustomDialog extends JDialog {
    private JPanel contentPane;
    private JButton buttonOK;
    private JButton buttonCancel;

    public CustomDialog() {
        setContentPane(contentPane);
        setModal(true);
        getRootPane().setDefaultButton(buttonOK);

        buttonOK.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                onOK();
            }
        });

        buttonCancel.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                onCancel();
            }
        });
    }
}
```

```

    });

    // call onCancel() when cross is clicked
    setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            onCancel();
        }
    });

    // call onCancel() on ESCAPE
    contentPane.registerKeyboardAction(new ActionListener() {
        public void actionPerformed(ActionEvent e) {
            onCancel();
        }
    }, KeyStroke.getKeyStroke(KeyEvent.VK_ESCAPE, 0), JComponent-
t.WHEN_ANCESTOR_OF_FOCUSED_COMPONENT);
}

private void onOK() {
    // add your code here
    dispose();
}

private void onCancel() {
    // add your code here if necessary
    dispose();
}

public static void main(String[] args) {
    CustomDialog dialog = new CustomDialog();
    dialog.pack();
    dialog.setVisible(true);
    System.exit(0);
}
}

```

Примечание:

В сгенерированном коде местами встречаются подсказки о том куда можно вставлять код с действиями если это нужно.

Теперь можно в целом модифицировать этот код для того, чтобы сделать своё настраиваемое диалоговое окно (На подобии **CAPTCHA**, только с кодом).

Для этого на форму добавляем 2 элемента **JTextField** и **JLabel**:

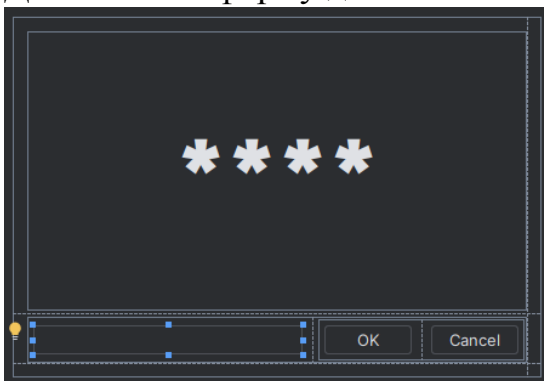


Рис. 5.1.3 JTextField (Выделен)

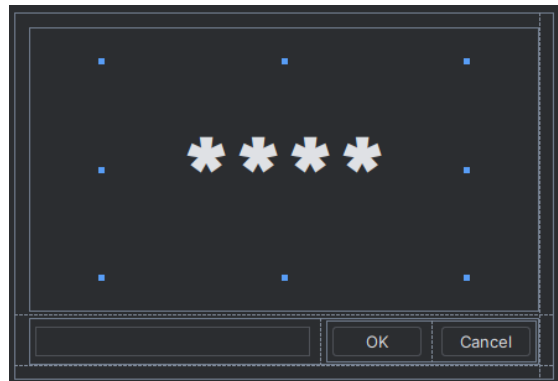


Рис. 5.1.4 JLabel (Выделен)

затем в свойствах ставим название (**field name**) для **JTextField** -> **codeCheckJTextField**, а для **JLabel** -> **randomCodeJLabel**.

Так же для **JLabel** меняем свойства как на рисунке 5.1.5:

Horizontal Align	Fill
Vertical Align	Fill
☒ Maximum Size	[300, 180]
width	300
height	180
font	Arial Black 72 pt Bold
text	****
verticalAlignment	Center
verticalTextPosition	Center

Рис. 5.1.5 свойства *JLabel*

После чего возвращаемся в код и добавляем в **import**'ы:

```
import java.util.Random;
```

В глобальные переменные:

```
private int generatedCode;
```

Затем изменяем блок, где комментарий:

```
// call onCancel() when cross is clicked
```

->

```
setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);  
addWindowListener(new java.awt.event.WindowAdapter() {  
    public void windowOpened(java.awt.event.WindowEvent e) {  
        onWindowOpened();  
    }  
  
    public void windowClosing(java.awt.event.WindowEvent e) {  
        onCancel();  
    }  
});
```

Так как мы добавили новый метод в слушатель, добавляем и сам этот метод:

```
private void onWindowOpened() {  
    // Генерация случайного 4-значного кода при открытии окна  
    Random random = new Random();  
    generatedCode = 1000 + random.nextInt(9000); // Генерируем случайное  
    число от 1000 до 9999  
    randomCodeJLabel.setText("" + generatedCode);  
}
```

Так же модифицируем метод **onOK()**:

```
try {  
    int enteredCode = Integer.parseInt(codeCheckJTextField.getText());  
    if (enteredCode == generatedCode) {  
        dispose();  
    } else {  
        onWindowOpened();  
    }  
} catch (NumberFormatException ex) {  
    JOptionPane.showMessageDialog(this, "Введите корректный код.");  
}
```

И добавляем в метод **main()** строчку после **dialog.pack()**:

```
dialog.setLocationRelativeTo(null);
```

И получается код:

```
import javax.swing.*;
import java.awt.event.*;
import java.util.Random;

public class CustomDialog extends JDialog {
    private JPanel contentPane;
    private JButton buttonOK;
    private JButton buttonCancel;
    private JTextField codeCheckJTextField;
    private JLabel randomCodeJLabel;
    private int generatedCode;

    public CustomDialog() {
        setContentPane(contentPane);
        setModal(true);
        getRootPane().setDefaultButton(buttonOK);

        buttonOK.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                onOK();
            }
        });

        buttonCancel.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                onCancel();
            }
        });

        // call onCancel() when cross is clicked
        setDefaultCloseOperation(DO_NOTHING_ON_CLOSE);
        addWindowListener(new java.awt.event.WindowAdapter() {
            public void windowOpened(java.awt.event.WindowEvent e) {
                onWindowOpened();
            }

            public void windowClosing(java.awt.event.WindowEvent e) {
                onCancel();
            }
        });

        // call onCancel() on ESCAPE
        contentPane.registerKeyboardAction(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                onCancel();
            }
        }, KeyStroke.getKeyStroke(KeyEvent.VK_ESCAPE, 0), JComponent-
t.WHEN_ANCESTOR_OF_FOCUSED_COMPONENT);
    }

    private void onWindowOpened() {
        // Генерация случайного 4-значного кода при открытии окна
        codeCheckJTextField.setText("");
        Random random = new Random();
        generatedCode = 1000 + random.nextInt(9000); // Генерируем случайное
число от 1000 до 9999
        randomCodeJLabel.setText("" + generatedCode);
    }

    private void onOK() {
        try {
            int enteredCode = Integer.parseInt(codeCheckJ-
TextField.getText());

```



```

        if (enteredCode == generatedCode) {
            dispose();
        } else {
            onWindowOpened();
        }
    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(this, "Введите корректный код.");
    }
}

private void onCancel() {
    // add your code here if necessary
    dispose();
}

public static void main(String[] args) {
    CustomDialog dialog = new CustomDialog();
    dialog.pack();
    dialog.setLocationRelativeTo(null);
    dialog.setVisible(true);
    System.exit(0);
}
}

```

Если все сделано правильно после запуска кода у нас появится диалоговое окно с случайным кодом:



Рис. 5.1.6 Итоговая форма

Логика работы программы – после открытия диалогового окна, генерируется случайный код из 4х цифр, если ввести этот код в **JTextField** то окно закроется, если код не совпадает, будет сгенерирован новый код.

Помимо этого, можно вызывать созданное диалоговое окно из других мест, к примеру создадим файл **Main** и вставим туда код:

```

import javax.swing.*;
import java.awt.*;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Main App");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(400, 300);

            Dimension screenSize = Toolkit.getDefaultToolkit().getScreenSize();

            int x = (screenSize.width - frame.getWidth()) / 2;
            int y = (screenSize.height - frame.getHeight()) / 2;

```

```

        frame.setLocation(x, y);

        JButton openDialogButton = new JButton("Проверить");
        openDialogButton.addActionListener(e -> {
            CustomDialog customDialog = new CustomDialog();
            customDialog.pack();
            customDialog.setVisible(true);
        });

        frame.add(openDialogButton);

        frame.setVisible(true);
    });
}
}

```

После запуска у нас создается форма с кнопкой Проверить, после нажатия на которую откроется наше диалоговое окно.

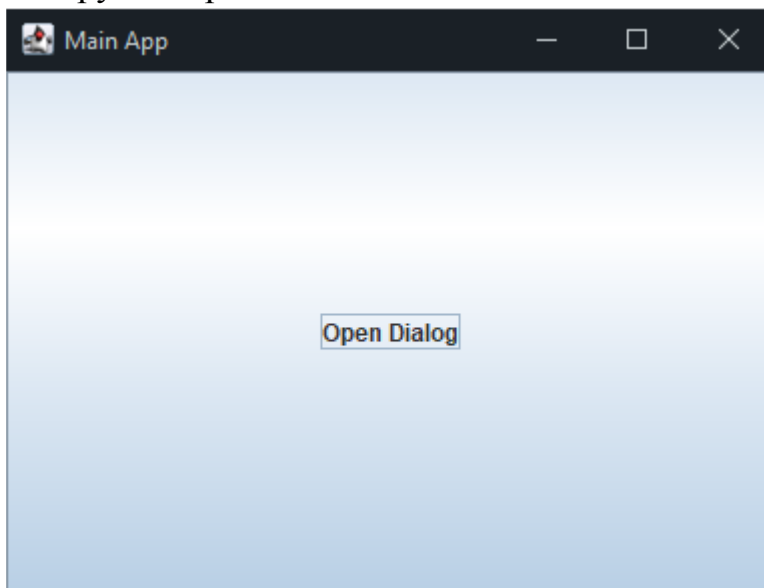


Рис. 5.1.7 Вызов из другой формы

Задание к лабораторной работе:

1. Сделать настраиваемое диалоговое окно из лабораторной работы.
2. Попробовать вызвать созданное диалоговое окно из другого места.
3. Создать своё собственное диалоговое окно.

Лабораторная работа №6

“Создание меню бара Swing (без конструктора)”

В Swing присутствует такой элемент интерфейса как **JMenu**, по какой-то причине в конструкторе (**Swing UI Designer**) нет данного элемента, но ничего не мешает закодировать элемент вручную.

Создаем проект, создаем любой класс, пусть будет название класса **Main**.

Вписываем код:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Main {
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            JFrame frame = new JFrame("Main Form");
            frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
            frame.setSize(400, 300);

            // Центрирование по экрану
            Dimension screenSize = Toolkit.getDefaultToolkit().getScreen-
Size();

            int x = (screenSize.width - frame.getWidth()) / 2;
            int y = (screenSize.height - frame.getHeight()) / 2;
            frame.setLocation(x, y);

            // Создание меню-бара
            JMenuBar menuBar = new JMenuBar();

            // Создание меню "File"
            JMenu fileMenu = new JMenu("File");
            JMenuItem exitMenuItem = new JMenuItem("Exit");

            // Добавление слушателя для пункта меню "Exit"
            exitMenuItem.addActionListener(new ActionListener() {
                @Override
                public void actionPerformed(ActionEvent e) {
                    System.exit(0);
                }
            });

            // Добавление пункта меню "Exit" в меню "File"
            fileMenu.add(exitMenuItem);

            // Создание меню "Edit"
            JMenu editMenu = new JMenu("Edit");
            JMenuItem cutMenuItem = new JMenuItem("Cut");
            JMenuItem copyMenuItem = new JMenuItem("Copy");
            JMenuItem pasteMenuItem = new JMenuItem("Paste");

            // Добавление пунктов в меню "Edit"
            editMenu.add(cutMenuItem);
            editMenu.add(copyMenuItem);
            editMenu.add(pasteMenuItem);

            // Добавление меню "File" и "Edit" в меню-бар
            menuBar.add(fileMenu);
            menuBar.add(editMenu);
        });
    }
}
```

```

        // Установка меню-бара в окне
        frame.setJMenuBar(menuBar);

        frame.setVisible(true);
    });
}
}

```

После запускаем и получаем вот такую форму:

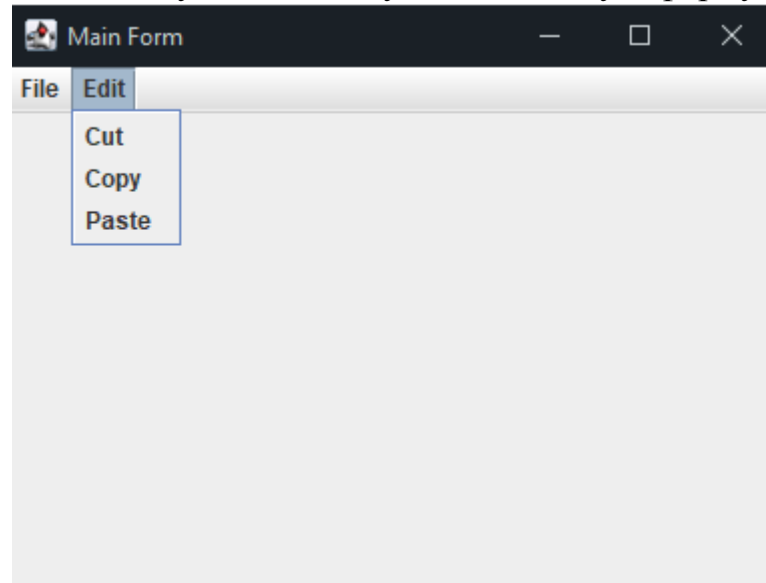


Рис. 6.1.1 Вызов из другой формы

А потом на примере этого кода можно увеличивать количество пунктов в меню баре и назначать на них какие-либо действия.

Пример создания слушателя и метода пунктов в меню бар:

```

// Добавление слушателя для пункта меню "Exit"
exitMenuItem.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.exit(0);
    }
});

```

Задание к лабораторной работе:

1. Создать форму с меню баром.
2. Основываясь на полученных знаниях за выполнение предыдущих лабораторных работ, добавить какие-либо действия для пунктов в меню баре.